* Batch Normalization also has been shown to reduce overfitting, and therefore many modern deep learning architectures don't use dropout at all.

Underfitting and Overfitting :-

* Underfitting is a scenario where a data model is unable to capture the relationship b/w the input and output variables accurately, generating a high error rate on both training set and unseen data.

* Underfitting occurs when a model is too simple, which can be result of a model needing more training time, more input features or less regularization

Overfitting

when the n/w tries to learn too much or too many details in the training data along with the noise from the training data which results in poor performance on unseen or test dataset when this happens the n/w fails to generalize the features / patterns

Scanned by TapScanner

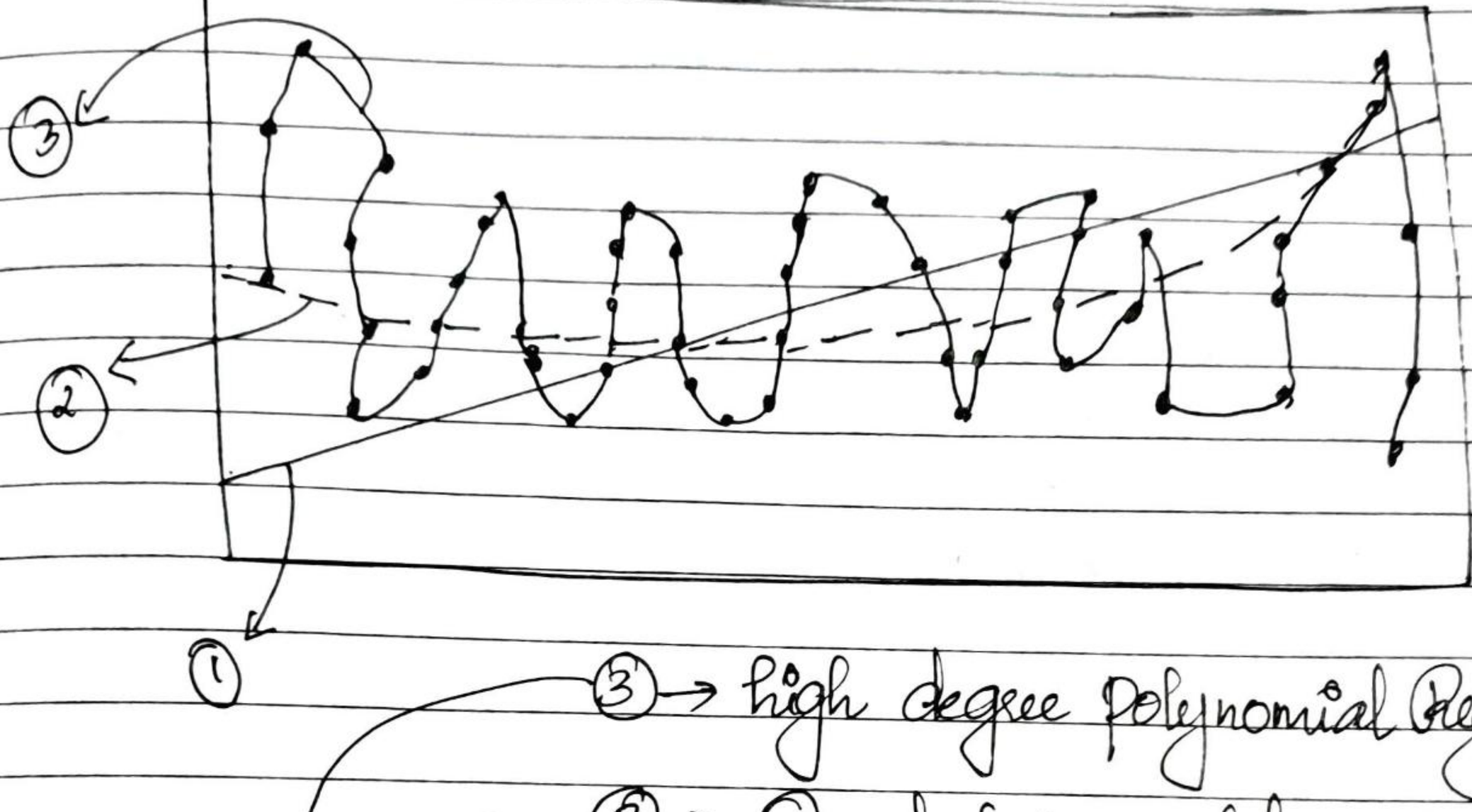
found in the training data.

* Overfitting during training can be spotted when the error on training data decreases to a very small value but the error on the new data or test data increases to a large value.

Scanned by TapScanner

Training and Validation Curves:

Eg for overfitting and underfitting problem:-



③ → high degree polynomial Regression

② → Quadratic model → generalized form

① → linear model

Overfitting

suffers from underfitting

- To find out model's generalization performance we can make use of cross-validation.
  and another important method is to look at the learning curves.

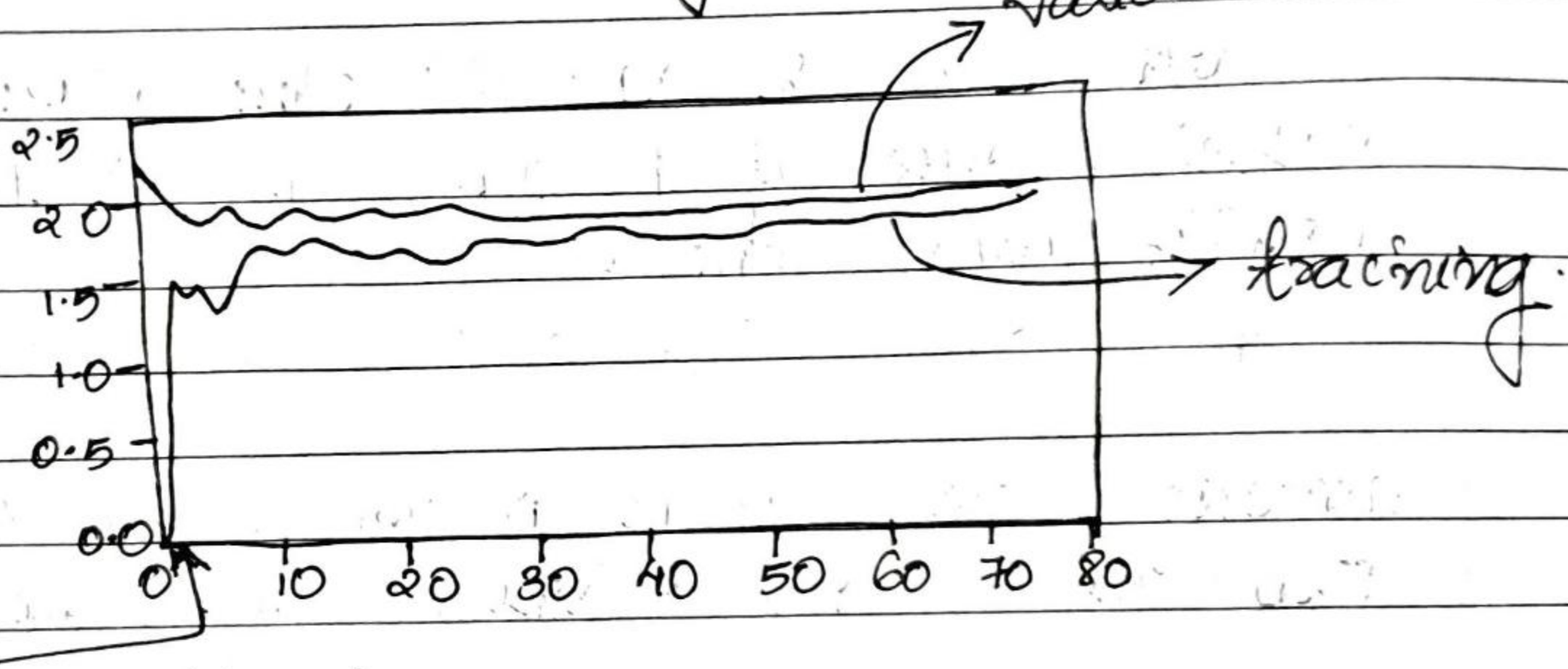
Scanned by TapScanner

Learning Curves:-

* Learning Curves are plots of the model's performance on the training set and the validation set as a function of training set size.

* To generate the plots, simply train the model several times on different sized subsets of the training set.

Let us consider an eg:-



* When there are just one or two instances in the training set the model can fit them perfectly, which is why the curve starts from zero.

* When new instances are added to the training set, it becomes impossible for the model to fit the

tracining data perfectly: because the data is noisy and bcz it is not linear at all.
∴ So the error on the tracining data goes cep centil it reaches a plateau, at which point adding new instances to the tracining set doesn't make the average error much better or worse

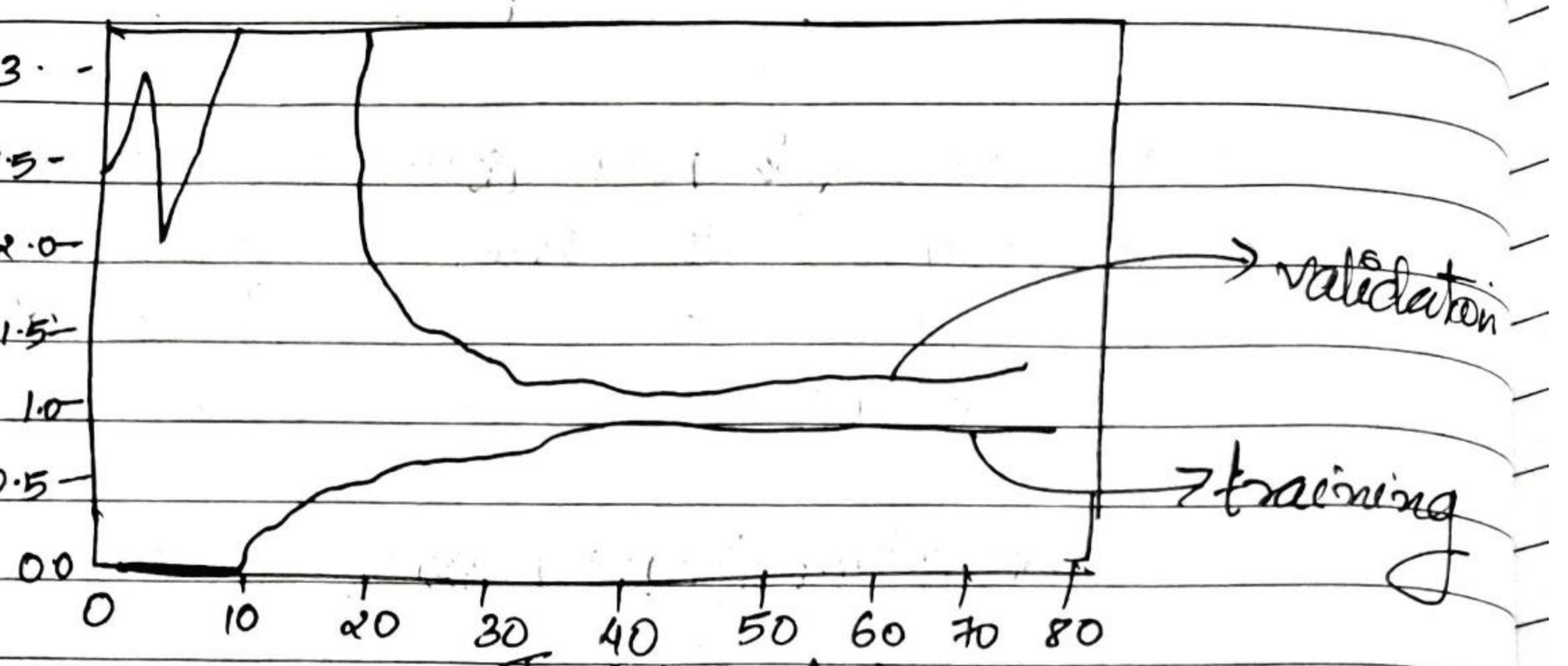
On Validation:-

When the model is trained on very few tracining instances, it is incapable of generalizing properly, which is why the validation error is initially quite big.

When with more tracining example, model learns and thus the validation error slowly goes down.

In this example, error ends cep at a plateau, very close to the other curve
This kind of model, from the learning curve we can understood its an underfitting model.

* If your model is underfitting the tracining data, adding more tracining examples will not help. We need to use a more complex model or come cep with better features *



* The error on the training data is much lower than

* Gap b/w the curves. This means that the model performs significantly better on the training data than on the validation data, called Overfitting

* When training set size gets increasing, the two curves would continue to get closer.

To improve an overfitting model is to feed it more training data until the validation error reaches the training error * *

URBAN EDGE ___/___/___

Bias / variance trade-off.

2 relavant topics: overfitting and underfitting.

Models generalization error can be expressed as the sum of three very different errors.

① Bias :-

* Generalization error is due to wrong assumptions
  eg:- assuming that the data is linear when it is actually quadratic.
* A high bias model is most likely to underfit the training data.

② Variance :-

* Due to model's excessive sensitivity to small variations in the training data.

* A model with many degrees of freedom, likely to have high variance;
  ↳ Overfit the training data

③ Irreducible Error :-

* Due to noisiness of the data itself.
* Only way to reduce this part of the error is to ~~clean~~ clean up the data: (eg:- detect and remove outliers)

Scanned by TapScanner

URBAN EDGE

Tradeoff

Model Complexity ↑ (increases)
↳ Variance (↑)
Bias (↓)

Models' Complexity (↓)
↳ Variance (↓)
Bias (↑)

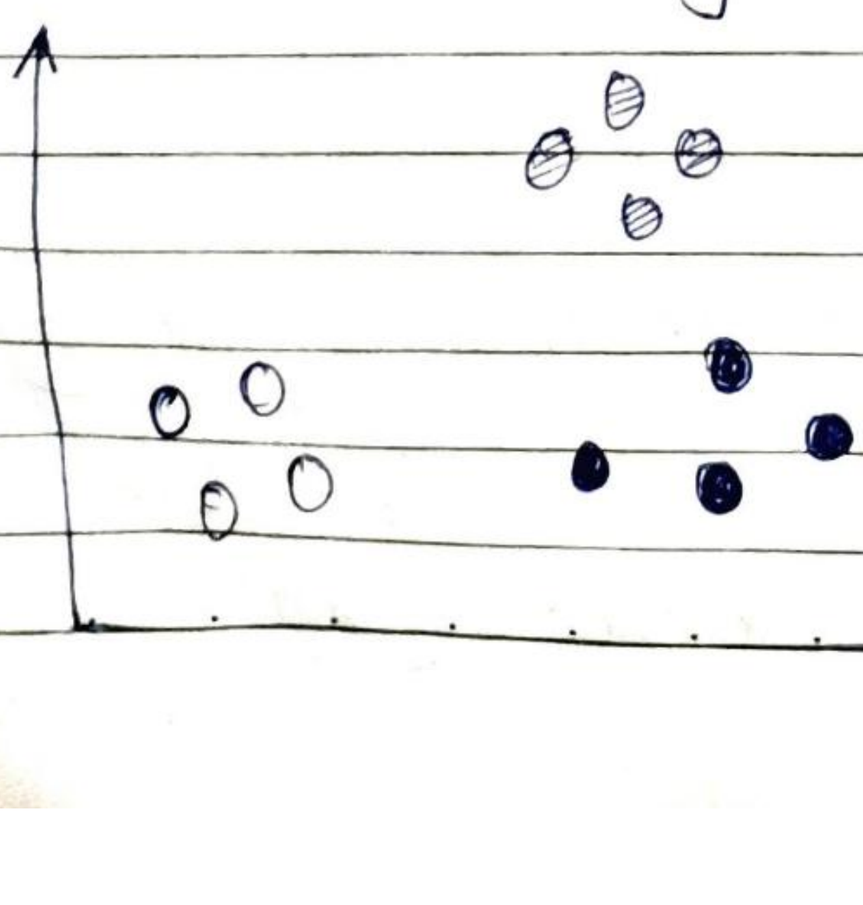
T-SNE - [t-distributed Stochastic Neighbour Embedding]

* Method for dimensionality reduction, used mainly for visualization of data in 2D or 3D maps.

(PCA - Linear Connections)

* This method can find non-linear connections in the data and therefore it is highly popular.

* Consider an example (2D)

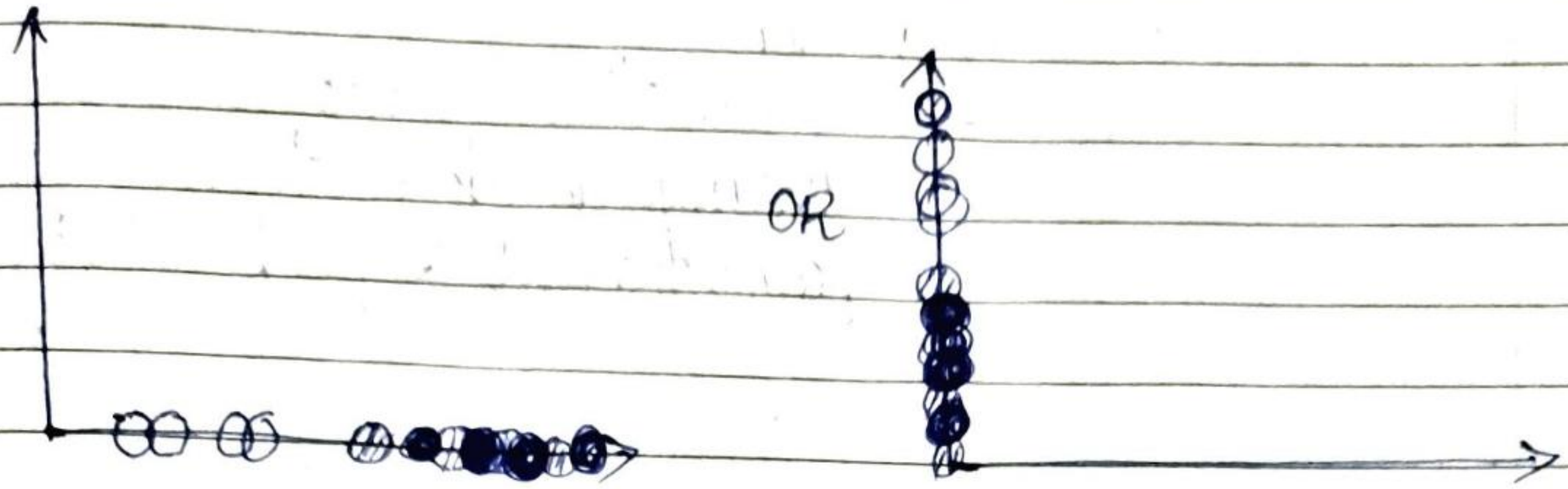


→ Each pattern represents a cluster
→ Each cluster has a different density

Scanned by TapScanner

URBAN EDGE

* Simply project the data onto (i below figure) one of its dimensions, we can see an overlap of atleast two of the clusters



= T-SNE algorithm deals with this problem !!!!

Using the below steps:-

① Calculating a joint probability distribution to represent the similarity b/w the data points.

② Creating a dataset of points in the target dimension and again calculate the joint probability of that distribution

③ Both joint probabilities should be similar in high dimensional data and in low dimensional data

Scanned by TapScanner

Applicable for original dataset

Algorithm:-

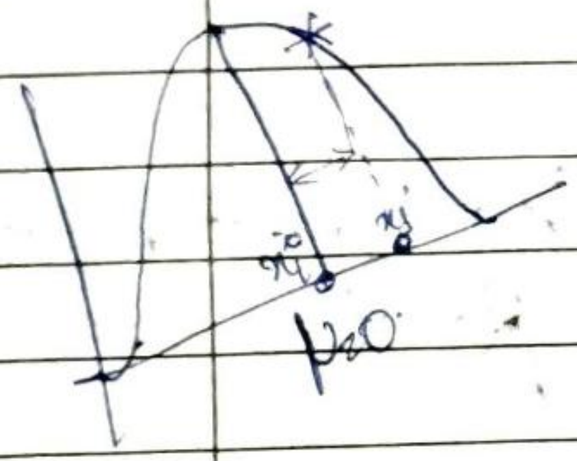
First Stage * Calculating the Euclidean distances of each point from all the other points.

- Take these distance and transforming them into conditional probability

↓

Calculating how similar every points in the data ie how likely they are to be neighbours.

Gaussian



* Conditional probability of point 'xj' given 'xi' is represented by a Gaussian centered at xi with a standard deviation of 'σi'

* ↳ Mathematically,

Gaussian

$$P_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)}$$

dividing by the sum of all the other points placed at the guassian centered 'xi' is that we may deal with clusters of different densities

and case about the densities of each clusters but to visualize them as clusters

Scanned by TapScanner

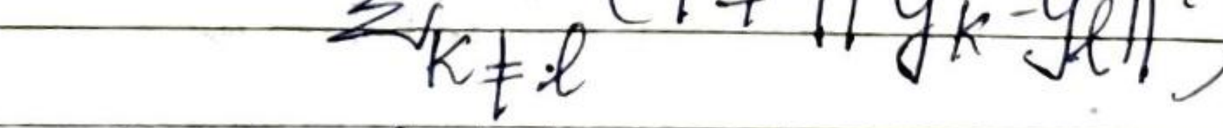
* From the conditional distributions, we need to calculate the joint probability distribution

$$P_{ij} = \frac{P_{j|i} + P_{i|j}}{2n}$$

$2n \rightarrow$ no: of datapoints

Second stage :- Creating data in a low dimension :-

*① Create a low dimensional space
*② Apply joint probability distribution for it.



Applying | calculating joint probability distribution using t-distribution instead of gaussian distribution

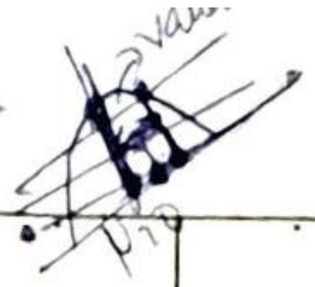
probability

$$q_{ij} = \frac{1 + \left(\|y_i - y_j\|^2\right)^{-1}}{\sum_{k \neq l} \left(1 + \|y_k - y_l\|^2\right)^{-1}}$$

Advantage of t-distribution :

① → Moderate distance b/w points in the high dimensional space to become extreme in the low dimensional space. prevents "crowding" of the points in the lower dimension.

Scanned by TapScanner



Q dest.

Third stage:-

Apply Kullback-Leibler divergence (KL divergence) → is a measure of how much two distributions (such as joint probability distribution of the data points in the low dimension and the original dataset) are different from one another.

→ Let P and Q are distribution on the probability space X, then 'KL' divergence

$$D_{KL}(P||Q) = \sum_{x \in X} P(x) \log\left(\frac{P(x)}{Q(x)}\right)$$

↳ when the distributions are similar
↳ value of KL divergence is smaller, reaching zero when the distributions are identical

Overall

** Changing lower dimension to similar to original dataset is done through gradient descent.

** Cost fn is calculating through KL divergence.

Scanned by TapScanner

URBAN EDGE ___/___/___

$$C = KL(P||Q) - \sum_i \sum_j p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

Parameters in the model

~~weight~~, ~~bias~~ * parameters for gradient descent like learning rate.

* Perplexity (for t-SNE)

used for choosing the std deviation (σ_i) of the Gaussian

Scanned by TapScanner